

Twisted — Reactor, Deferreds, Protocols

The asynchronous engine the whole node runs on — the reactor that is Hathor's event loop, the Deferred that is its future, and the protocol/factory pattern behind every connection.

SUBJECT hathor-core · Part I · Track C (the stack)

CHAPTER 16 · Foundations · The Stack

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Twisted · Reactor · Deferred · addCallback / errback · inlineCallbacks · Protocol · Factory · Endpoints · LoopingCall · deferToThread · asyncio reactor

Table of Contents

Chapter 16 — Twisted: Reactor, Deferreds, Protocols	3
16.1 What Twisted is	4
16.2 The problem it solves	4
16.3 The core model and its components	5
16.4 The reactor — the one loop	6
How hathon-core obtains and runs the reactor	7
16.5 The Deferred — Twisted's future	8
A tiny generic Deferred	8
maybeDeferred — "synchronous or asynchronous, I don't care"	9
inlineCallbacks — straight-line asynchronous code	9
Deferreds in hathon-core	9
16.6 Protocols, factories, endpoints, timers	10
16.6.1 The connection lifecycle, in callbacks	11
16.6.2 A tiny generic Protocol and Factory	11
16.6.3 Hathor's Protocol and Factory	12
16.6.4 Endpoints, timers, and the thread-pool escape hatch	14
16.7 Why Twisted and not the alternatives	16
Twisted vs. asyncio	16
The typed-reactor abstraction (why the wrapper exists at all)	17
And the thread question	18
Recap	19

Chapter 16 — Twisted: Reactor, Deferreds, Protocols

What you'll learn in this chapter

- What **Twisted** is — the asynchronous-networking framework `hathor-core` is built on — and the concrete problem it solves that plain Python does not.
- The **reactor**: Twisted's name for the single event loop the entire node runs on, and what `reactor.run()` actually does.
- The **Deferred**: Twisted's name for a future, and how `addCallback` / `addErrback` chains, `maybeDeferred`, and `@inlineCallbacks` tame asynchronous code.
- The **Protocol** and **Factory** pattern — the two objects behind every network connection — and why the callbacks `connectionMade`, `dataReceived`, and `connectionLost` are the backbone of the P2P layer.
- The supporting cast: **endpoints**, the **LoopingCall** timer, **callLater**, and **deferToThread** / the thread-pool escape hatch for CPU-bound work.
- **Why Twisted and not asyncio** — the honest trade-off — and how Hathor's own `hathor/reactor/` abstraction keeps the door open to an `asyncio`-backed reactor via `--x-asyncio-reactor`.

Chapter 2 built the ideas of asynchronous programming on plain Python: the event loop, callbacks, futures, `async` / `await`, the never-block rule, and the thread-pool escape hatch. It ended with a deliberate promissory note — its Bridge (\$2.8) mapped each idea onto a Twisted name and pointed here. This chapter pays that note off. Every concept you met in Chapter 2 has a Twisted spelling, and this is where we learn it on the real framework and the real `hathor-core` code.

Twisted is not background detail. It is the most all-touching dependency in the codebase: the node is a Twisted program. Boot ends with one call into Twisted (`reactor.run()`), and from that instant Twisted owns the one thread the whole node lives on. If you internalize this chapter, a great deal of the codebase's shape — small methods that return fast, objects whose methods are named like events, the conspicuous absence of `time.sleep` — stops being mysterious.

We follow the technology-primer template: what it is, the problem it solves, its core components, tiny generic examples, then the real cited Hathor usage, and finally the trade-off against the alternative (`asyncio`).

16.1 What Twisted is

Twisted is an event-driven networking framework for Python. Two words in that sentence carry the weight.

Networking — Twisted's reason to exist is writing programs that talk to many other computers at once: servers, clients, peer-to-peer nodes. It ships mature, tested implementations of the protocols such programs need (TCP, TLS, HTTP, DNS, and more), so you do not re-write socket plumbing for every project.

Event-driven — Twisted does not run your program top-to-bottom. You register handlers for things that might happen — "a connection arrived," "bytes are ready to read," "this timer expired" — and hand control to Twisted's central loop, which waits for those events and calls your handlers when they occur. This is exactly the event-loop model of Chapter 2, packaged as a framework.

Recap — the event loop (full treatment in Ch. 2). An event loop is a `while` loop that does one thing forever: take the next ready piece of work, run it to completion, repeat. It lets a single thread make progress on thousands of tasks by switching between them whenever one would otherwise wait. Twisted's event loop is called the **reactor**. → full treatment in Ch. 2 (§2.4 builds one from scratch).

Twisted is old by software standards — it predates Python's own `asyncio` by more than a decade. That age is mostly an asset (its protocol implementations have been battle-tested in production for years) and partly a liability (it has its own vocabulary, written before `async / await` existed). We name both sides honestly in §16.7.

The category is worth fixing in your mind: Twisted is to asynchronous networking what a web framework is to HTTP request-handling — a body of reusable machinery plus a set of conventions you write your code into. You do not call Twisted so much as Twisted calls you. That inversion — "don't call us, we'll call you" — is the defining feel of the framework, and it follows directly from the event-driven model.

16.2 The problem it solves

Recap — the blocking problem (full treatment in Ch. 2, §2.1). A blocking¹ call (a naive `socket.recv`, a `time.sleep`) freezes the calling thread until it finishes. A node holds thousands of network connections at once; if reading from one slow peer froze the whole program, no other peer could be served. The classic fixes are one-thread-per-connection (memory-heavy, and shared memory invites race conditions and locks) or a single-threaded **event loop** that never blocks. A node is overwhelmingly I/O-bound² — it spends its life waiting on slow conversations — so the event-loop model fits it best. → full treatment in Ch. 2.

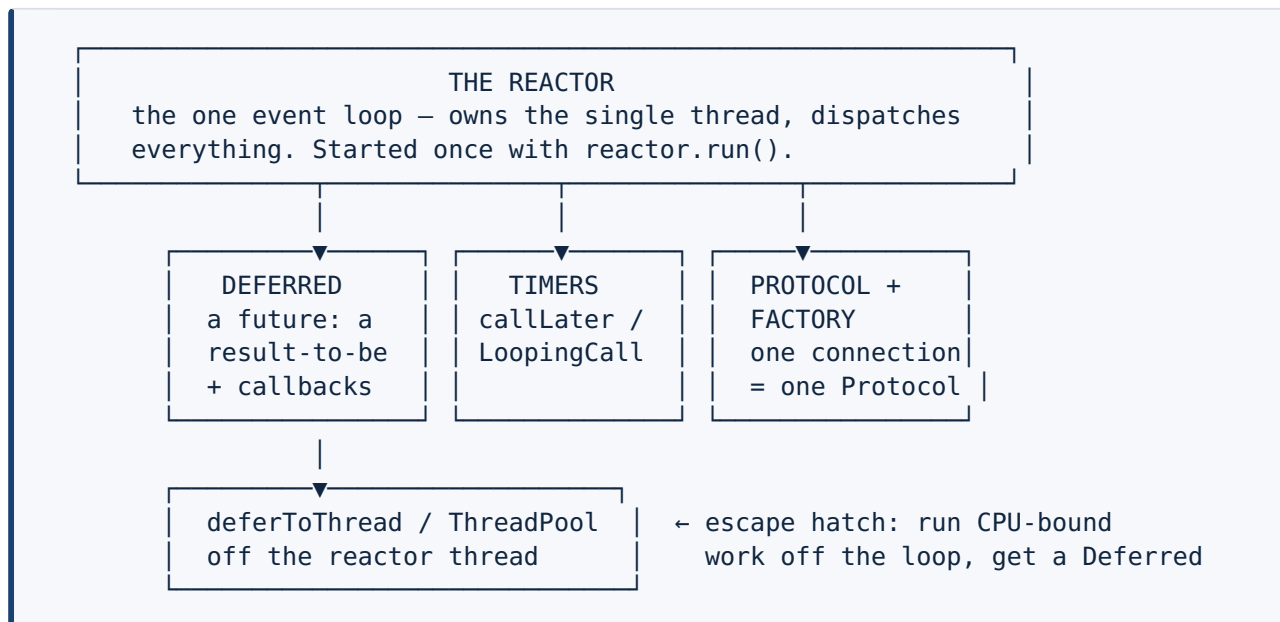
Concretely, without a framework like Twisted you would have to write, by hand:

1. **The loop itself** — the `while` loop that asks the operating system "which of my thousands of sockets have data right now?" (via a system call like `epoll / select`) and dispatches to the right handler.
2. **A timer subsystem** — "run this callback in 30 seconds," kept sorted by due-time and checked every iteration.
3. **Protocol parsers** — turning a raw, arbitrarily-chunked byte stream into whole messages (TCP gives you bytes, not messages; a single `recv` may return half a message or three).
4. **TLS, DNS, connection-retry logic, graceful shutdown** — each a project in itself.

Twisted provides all of this, correct and tested, and asks only that you express your logic as handlers it can call. The problem it solves, in one line: **it lets you write a high-concurrency network program as a set of small callbacks on one thread, without hand-rolling the event loop, the timers, and the protocol plumbing.**

16.3 The core model and its components

Twisted has a small number of central abstractions. Learn these six names and you can read most of the asynchronous code in `hathor-core`:



- **The reactor** — the event loop. There is exactly one per process. It owns the single thread; everything else is something the reactor calls or something that gives the reactor more work. (§16.4)
- **The Deferred** — Twisted's future: an object standing in for a result that is not ready yet, to which you attach callbacks. (§16.5)

- **Timers** — `reactor.callLater(delay, fn)` runs `fn` once after a delay; `LoopingCall` runs a function repeatedly on a fixed interval. (§16.6.4)
- **Protocol** — an object representing one connection; the reactor calls its methods (`dataReceived`, ...) when events happen on that connection. (§16.6.2)
- **Factory** — an object that manufactures a fresh Protocol instance for each new connection, and holds the state shared across connections. (§16.6.3)
- **Endpoints / deferToThread** — endpoints describe "where to listen or connect"; `deferToThread` pushes a blocking or CPU-bound job onto a worker thread and hands you a Deferred for its result. (§16.6.4)

The rest of the chapter takes these one at a time: generic Twisted first, then Hathor's real code.

16.4 The reactor — the one loop

The reactor is the production version of the `MiniLoop` you built in Chapter 2. It is a single object, created once, that runs the event loop for the whole process. You obtain it, schedule some initial work, and then hand it the thread:

```
from twisted.internet import reactor

def say_hello():
    print("the reactor is running now")

reactor.callWhenRunning(say_hello)  # schedule work for once the loop starts
reactor.run()                       # HAND OVER THE THREAD — blocks here forever
print("this line runs only after the reactor stops")
```

`reactor.run()` is the line that matters. Before it, you are in ordinary top-to-bottom Python: importing, configuring, wiring objects together. `reactor.run()` starts the loop and **does not return** until the reactor is stopped — from that call onward, your program is reactive. It sleeps until an event (a connection, readable bytes, an expired timer) wakes it, runs the matching callback to completion, and sleeps again. This is the single most consequential fact about the structure of a Twisted program: there is a clear "before" (synchronous setup) and "after" (the reactive event loop), and the boundary is `reactor.run()`.

A few reactor methods recur:

- `reactor.callWhenRunning(fn)` — run `fn` once, as soon as the loop is up.
- `reactor.callLater(delay, fn)` — run `fn` once, `delay` seconds from now (a one-shot timer).
- `reactor.listenTCP(port, factory)` — start accepting connections; build a Protocol (via the factory) for each.
- `reactor.connectTCP(host, port, factory)` — open an outgoing connection.

- `reactor.addSystemEventTrigger('after', 'shutdown', fn)` — run `fn` during clean shutdown.
- `reactor.stop()` — stop the loop; `reactor.run()` then returns.

That last pair shows up in Hathor at once: the manager registers its own `stop` to run during reactor shutdown (`hathor/manager.py:157`), so a clean shutdown of the loop tears the node down in order.

How `hathor-core` obtains and runs the reactor

Hathor does not import the global Twisted reactor scattered across the codebase. It wraps reactor access behind its own small package, `hathor/reactor/`, for two reasons we unpack in §16.7: to give the reactor a precise type (Twisted's reactor is typed loosely), and to allow swapping in an `asyncio`-backed reactor. The package exposes two functions (`hathor/reactor/__init__.py:15`):

```
from hathor.reactor.reactor import get_global_reactor, initialize_global_reactor
from hathor.reactor.reactor_protocol import ReactorProtocol
```

`initialize_global_reactor` is called exactly once, early in boot, and `get_global_reactor` hands the same instance to everyone afterward (`hathor/reactor/reactor.py:29`):

```
def get_global_reactor() -> ReactorProtocol:
    global _reactor
    if _reactor is None:
        raise Exception('The reactor is not initialized. Use `initialize_global_reactor()`.')
    return _reactor
```

Recap — the singleton (full treatment in Ch. 3, §3.4). A singleton is a pattern guaranteeing one shared instance of something, reached through a global accessor. The module-level `_reactor` plus `get_global_reactor()` is exactly that — and Twisted's reactor is inherently a singleton (there can be only one event loop per process). Chapter 3 flagged the global-state smell; here it is unavoidable, because the loop is singular. → full treatment in Ch. 3.

The actual boot path lives in the `run_node` command. It initializes the reactor with the `asyncio` flag drawn from the command line (`hathor_cli/run_node.py:202`):

```
from hathor.reactor import initialize_global_reactor
reactor = initialize_global_reactor(use_asyncio_reactor=self._args.x_asyncio_reactor)
```

and, at the very end of startup — after the whole node has been assembled and the manager started — hands over the thread (`hathor_cli/run_node.py:592`):

```
self.reactor.run()
```

That single line is the hinge of the entire node lifecycle. Everything in Part II up to Chapter 33 is setup that happens before it; everything the node does in steady state happens inside the loop it starts. When Chapter 0 said "the reactor is put in gear," this is the gear.

16.5 The Deferred — Twisted's future

Recap — the future (full treatment in Ch. 2, §2.5). A future is an object representing "a result that isn't ready yet." Instead of blocking until a value arrives, an asynchronous function returns a future immediately; you attach a callback that fires when the value is ready, and you can chain callbacks so each step feeds the next. This tames the nested "pyramid of doom" of raw callbacks. Twisted's name for a future is the **Deferred**. → full treatment in Ch. 2.

A `Deferred` is an object that says: "I don't have the result yet. Here is where you attach the code to run when I do — and here is where you attach the code to run if it fails instead." Those two halves are the heart of the design. A `Deferred` carries **two chains of callbacks**:

- the **callback** chain — run on success, each receiving the previous one's return value;
- the **errback** chain — run on failure, each receiving a `Failure`³ (Twisted's wrapper around an exception).

A tiny generic Deferred

```
from twisted.internet.defer import Deferred

def got_result(value):
    print("success:", value)
    return value * 2          # whatever I return feeds the NEXT callback

def got_error(failure):
    print("something failed:", failure.getErrorMessage())

d = Deferred()
d.addCallback(got_result)    # attach a success handler
d.addErrback(got_error)     # attach a failure handler

# ... later, when the result is ready, somebody "fires" the Deferred:
d.callback(21)              # prints "success: 21", and got_result returns 42
```

Two things to notice. First, you attach handlers before the result exists; firing the `Deferred` (`d.callback(21)`) is what triggers them. Second, callbacks **chain**: `got_result` returns `42`, and that value would be passed to the next `addCallback` if one were attached. This is how you express "read A, then transform A, then store the result" as a pipeline rather than a pyramid:

```
d.addCallback(parse)           # step 1: parse the raw bytes
d.addCallback(validate)       # step 2: validate the parsed object (gets parse's result)
d.addCallback(store)          # step 3: store it (gets validate's result)
d.addErrback(log_failure)     # if ANY step above raised, jump here
```

If any callback raises, control skips the remaining callbacks and jumps to the next errback — the same way a `try / except` skips to its handler. This unified success-and-error chaining is the Deferred's real contribution over a bare callback.

maybeDeferred — "synchronous or asynchronous, I don't care"

A recurring problem: a handler you call might finish immediately (return a plain value) or might kick off asynchronous work (return a Deferred). You would rather not branch on which. `maybeDeferred(fn, *args)` solves this: it calls `fn`, and whatever happens, you get back a Deferred — already-fired if `fn` returned a value or raised, still-pending if `fn` returned a Deferred. The caller treats every case uniformly.

inlineCallbacks — straight-line asynchronous code

Chaining `.addCallback(...)` still has visible plumbing. Twisted's answer — predating `async / await` — is the `@inlineCallbacks` decorator, which lets you `yield` a Deferred and write asynchronous code that reads top-to-bottom:

```
from twisted.internet.defer import inlineCallbacks

@inlineCallbacks
def load_all():
    a = yield read_async("a")    # pause here until the Deferred fires; loop runs others
    b = yield read_async("b")    # then pause here
    print(a, b)                 # both ready, in order, no nesting
```

Read `yield someDeferred` as exactly the `await` of Chapter 2: "pause this function here, hand the one thread back to the reactor so it can run other work, and resume me right here with the result once the Deferred fires." `@inlineCallbacks` is Twisted's coroutine mechanism — the same idea as `async / await`, spelled with `yield` because it was built before Python had the newer keywords. (Inside an `@inlineCallbacks` function on older Python you "returned" a value with `returnValue(x)`; on modern Python a plain `return x` works, so `returnValue` is legacy you may still encounter.)

Recap — decorators (full treatment in Ch. 4). A decorator wraps a function to give it new behavior without rewriting it: `@inlineCallbacks` transforms an ordinary generator function into one the reactor can drive step-by-step, pausing at each `yield`. You met the "interpose, don't rewrite" idea, and `@inlineCallbacks` specifically, in Chapter 4's Bridge. → full treatment in Ch. 4.

Deferreds in `hathor-core`

The manager imports the Deferred machinery directly (`hathor/manager.py:23`):

```
from twisted.internet import defer
from twisted.internet.defer import Deferred
```

The cleanest real example is the P2P protocol's message dispatcher. When a message arrives, `recv_message` looks up a handler and runs it — but a handler may finish synchronously or asynchronously, so it wraps the call in `maybeDeferred` and chains a success callback and an errback (`hathor/p2p/protocol.py:322`):

```
def recv_message(self, cmd: ProtocolMessages, payload: str) -> None:
    """ Executed when a new message arrives. """
    assert self.state is not None
    now = self.reactor.seconds()
    self.last_message = now
    ...
    cmd_handler = self.state.cmd_map.get(cmd)
    if cmd_handler is None:
        ...
        self.send_error_and_close_connection('Invalid Command: {} {}'.format(cmd, payload))
        return

    deferred_result: Deferred[None] = defer.maybeDeferred(cmd_handler, payload)
    deferred_result \
        .addCallback(lambda _: self.reset_idle_timeout()) \
        .addErrback(self._on_cmd_handler_error, cmd)
```

This one method shows the Deferred pattern at work. `maybeDeferred(cmd_handler, payload)` (:342) turns "synchronous or asynchronous handler" into one shape — always a Deferred. Then a callback/errback pair is chained onto it (:343 - :345): on success, reset the connection's idle timer; on failure, jump to `_on_cmd_handler_error` — a single place that catches whatever any command handler raised, instead of a `try/except` around every command. The errback reads the original exception back out of the boxed `Failure` (`hathor/p2p/protocol.py:347`):

```
def _on_cmd_handler_error(self, failure: Failure, cmd: ProtocolMessages) -> None:
    self.log.error(f'recv_message processing error:\n{failure.getTraceback()}',
                  reason=failure.getErrorMessage())
    self.send_error_and_close_connection(f'Error processing "{cmd.value}" command')
```

`failure.getTraceback()` and `failure.getErrorMessage()` are how you recover the traceback and message from the `Failure` box that travelled down the errback chain. Note the extra argument: `addErrback(self._on_cmd_handler_error, cmd)` passes `cmd` along, so the failure handler knows which command failed — extra positional args to `addCallback / addErrback` are forwarded to the handler after the result/failure.

16.6 Protocols, factories, endpoints, timers

This section covers the four pieces that turn the reactor into an actual network node. Each gets a generic Twisted sketch, then Hathor's real usage.

16.6.1 The connection lifecycle, in callbacks

Recall the central inversion: Twisted calls you. A network connection in Twisted is represented by a **Protocol** object whose methods are callbacks the reactor invokes as the connection's life unfolds:

```
reactor accepts/opens a connection
    |
    ▼
connectionMade()      ← "you're connected; set things up"
    |
    ▼ (bytes arrive, possibly many times)
dataReceived(bytes)   ← "here are some bytes that just arrived"
dataReceived(bytes)
    |
    ▼
connectionLost(reason) ← "the connection ended; clean up"
```

These three method names — `connectionMade`, `dataReceived`, `connectionLost` — are the §2.3 callback pattern made the backbone of every connection. You do not call them; you implement them, and the reactor calls them at the right moments.

16.6.2 A tiny generic Protocol and Factory

```
from twisted.internet.protocol import Protocol, Factory

class Echo(Protocol):                                # ONE connection
    def connectionMade(self):
        print("a peer connected")
    def dataReceived(self, data):                    # called each time bytes arrive
        self.transport.write(data)                 # echo them straight back
    def connectionLost(self, reason):
        print("peer gone:", reason)

class EchoFactory(Factory):                          # MAKES one Echo per connection
    def buildProtocol(self, addr):
        return Echo()
```

Two objects, two responsibilities — and this division is the point:

- A **Protocol** instance handles **one** connection. Its per-connection state (buffers, who the peer is, how far a handshake has progressed) lives on `self`. A new one is created for every connection and discarded when that connection closes. `self.transport` is the object you call to send bytes back out.
- A **Factory** is created **once** and makes a fresh Protocol for each incoming connection via `buildProtocol(addr)`. Shared, cross-connection state (a registry of all peers, configuration, references to the rest of the application) lives on the factory, so every Protocol it builds can reach it.

Recap — the factory pattern (full treatment in Ch. 3, §3.1). A factory is an object whose job is to create other objects, centralizing the "how to build one" decision. Twisted's `Factory.buildProtocol` is a textbook example: the reactor says "a connection arrived, give me a handler," and the factory manufactures one — wiring in whatever shared dependencies the new Protocol needs. → full treatment in Ch. 3.

You start a server by handing the factory to the reactor:

```
from twisted.internet import reactor
reactor.listenTCP(8000, EchoFactory())    # accept connections on port 8000
reactor.run()
```

`listenTCP` tells the reactor to accept TCP connections on the port; for each one, it calls `EchoFactory().buildProtocol(addr)`, attaches a transport, and starts delivering events to that Protocol's callbacks.

16.6.3 Hathor's Protocol and Factory

Hathor's per-connection object is `HathorProtocol`, and its docstring states the one-per-connection rule outright (`hathor/p2p/protocol.py:51`):

```
class HathorProtocol:
    """ Implements Hathor Peer-to-Peer Protocol. An instance of this class is
        created for each connection.
    """
    ...
```

A note on layering. Raw TCP delivers an arbitrarily chunked byte stream — a single `dataReceived` call might hand you half a message, or two-and-a-half messages. Hathor does not parse that by hand; it builds on Twisted's `LineReceiver`, which buffers the stream and calls back once per complete line. The concrete class combines Hathor's logic with that Twisted base (`hathor/p2p/protocol.py:430`):

```
class HathorLineReceiver(LineReceiver, HathorProtocol):
    """ Implements HathorProtocol in a LineReceiver protocol.
        It is a TCP connection which sends one message per line. """
    def connectionMade(self) -> None:
        super(HathorLineReceiver, self).connectionMade()
        self.setLineMode()
        self.on_connect()                                # Hathor's per-connection setup

    def connectionLost(self, reason: Failure = connectionDone) -> None:
        super(HathorLineReceiver, self).connectionLost()
        self.on_disconnect(reason)                       # Hathor's per-connection cleanup

    def lineReceived(self, line: bytes) -> None:         # one whole message at a time
        ...
        self.recv_message(cmd, msgdata)                 # → the dispatcher of §16.5
```

This is multiple inheritance used as composition of behaviors: `LineReceiver` supplies the buffer-the-stream-into-whole-lines machinery; `HathorProtocol` supplies the Hathor handshake and message logic. The lifecycle hooks are overridden to do both — `connectionMade` (:436) calls up to `LineReceiver` and then runs Hathor's `on_connect`, and `connectionLost` (:441) does the same with `on_disconnect`. Crucially, Hathor does not implement raw `dataReceived` at all; `LineReceiver` does, re-assembling the chunked byte stream into whole lines and calling `lineReceived` (:450) once per complete message — which parses the command and routes to the `recv_message` dispatcher of §16.5 (:477). The reactor sees a Protocol; Hathor sees its own peer object; Twisted's base class bridges the two.

The matching factory lives in `hathor/p2p/factory.py`. A shared base implements `buildProtocol` to manufacture a fresh `HathorLineReceiver` per connection, wiring in the dependencies every connection needs (`hathor/p2p/factory.py:43`):

```
class _HathorLineReceiverFactory(ABC, protocol.Factory):
    inbound: bool

    def buildProtocol(self, addr: IAddress) -> HathorLineReceiver:
        p = HathorLineReceiver(
            my_peer=self.my_peer,          # this node's own identity    (shared)
            p2p_manager=self.p2p_manager,  # the connections manager    (shared)
            use_ssl=self.use_ssl,
            inbound=self.inbound,
            settings=self._settings,
        )
        p.factory = self                  # give the Protocol a handle back to its factory
        return p
```

Two concrete factories subclass it — one for connections we accept, one for connections we open (`hathor/p2p/factory.py:55`, :61):

```
class HathorServerFactory(_HathorLineReceiverFactory, protocol.ServerFactory):
    inbound = True          # a peer connected to us

class HathorClientFactory(_HathorLineReceiverFactory, protocol.ClientFactory):
    inbound = False         # we connected to a peer
```

The split is the textbook one: shared, cross-connection state (`my_peer`, the `ConnectionsManager`, TLS settings) lives on the factory, created once; per-connection state lives on each `HathorProtocol`, created fresh by `buildProtocol`. The single `inbound` flag is all that distinguishes "they called us" from "we called them" — useful because the handshake differs slightly by direction. We meet this whole machine properly — the handshake, the peer state machine, sync-v2 — in **Chapters 34-35**; here it is the canonical illustration of the Protocol/Factory split.

16.6.4 Endpoints, timers, and the thread-pool escape hatch

Endpoints. `reactor.listenTCP` / `connectTCP` are the low-level way to start listening or connecting. Twisted also offers endpoints — small objects that describe "where" (a TCP server on port 8000, a TLS client to `host:port`) independently of the act of using them. They are a thin convenience layer over the same reactor calls; the typed reactor abstraction Hathor uses exposes `listenTCP` and `connectTCP` directly (`hathor/reactor/reactor_tcp_protocol.py:33`, `:43`), and the P2P layer builds its connections on top.

Timers — `callLater` and `LoopingCall`. A node needs both one-shot deadlines and repeating background work. Twisted offers two timer forms:

- `reactor.callLater(delay, fn)` — fire `fn` once, after `delay` seconds.
- `LoopingCall(fn)` — fire `fn` repeatedly, every N seconds, until stopped.

Hathor uses `callLater` for one-shot deadlines, such as the per-connection idle timeout — if a peer goes quiet, fire `on_idle_timeout` later (`hathor/p2p/protocol.py:247`):

```
self._idle_timeout_call_later = self.reactor.callLater(self.idle_timeout, self.on_idle_timeout)
```

Generic `LoopingCall`:

```
from twisted.internet.task import LoopingCall

def heartbeat():
    print("still alive")

lc = LoopingCall(heartbeat)
lc.start(5.0, now=False)  # run heartbeat() every 5 seconds; don't fire immediately
```

Hathor uses exactly this to poll its sync state. The manager creates the `LoopingCall` in its constructor (`hathor/manager.py:251`):

```
self.lc_check_sync_state = LoopingCall(self.check_sync_state)
self.lc_check_sync_state.clock = self.reactor
self.lc_check_sync_state_interval = self.CHECK_SYNC_STATE_INTERVAL
```

and starts it when the manager starts (`hathor/manager.py:331`):

```
self.lc_check_sync_state.start(self.lc_check_sync_state_interval, now=False)
```

Two details worth pausing on. The `clock = self.reactor` assignment (`:252`) makes the timer read its sense of "now" from the reactor rather than the wall clock — which is what lets the **simulator** (Chapter 43) advance time deterministically in tests by stepping a fake reactor's clock. And `now=False` means "wait one full interval before the first run," not "fire immediately." The polled method itself, `check_sync_state` (`hathor/manager.py:933`), reads the current time from `self.reactor.seconds()` (`:935`) — the node asks the reactor for the time, never the OS, so everything stays consistent under simulation.

Recap — never block the loop (full treatment in Ch. 2, §2.7). The reactor runs one callback at a time, to completion. So any callback that blocks — `time.sleep`, a synchronous network or disk call, a long CPU crunch — freezes every connection, timer, and task until it returns. The rule is absolute: never block the reactor. Each callback must do a little and return fast. → full treatment in Ch. 2.

The thread-pool escape hatch — CPU-bound work off the loop. The never-block rule has one awkward exception: genuinely CPU-bound work that cannot be made to yield. Proof-of-work⁴ hashing is the canonical case — it must grind a hash function as fast as possible, with no I/O to yield on. Running it on the reactor thread would freeze the node. The escape hatch (Chapter 2, §2.7) is to push that work onto a **thread pool**⁵ of worker threads and get a Deferred for its result, leaving the reactor free.

The manager creates a dedicated pool for exactly this (`hathor/manager.py:234`):

```
# Thread pool used to resolve pow when sending tokens
self.pow_thread_pool = ThreadPool(minthreads=0, maxthreads=settings.MAX_POW_THREADS, name='Pow th
```

It is started when the node starts (`hathor/manager.py:310`) and stopped on shutdown (`hathor/manager.py:356`):

```
self.pow_thread_pool.start()
...
if self.pow_thread_pool.started:
    self.pow_thread_pool.stop()
```

The generic Twisted way to send one job to a pool and get a Deferred back is `deferToThread`:

```
from twisted.internet.threads import deferToThread

d = deferToThread(expensive_hash, block_header) # runs on a worker thread
d.addCallback(on_hash_found)                   # fires back on the reactor thread
```

The crucial property: `deferToThread` runs the function on a worker thread, but the Deferred's callbacks fire back **on the reactor thread** — so your success handler is back in the safe single-threaded world, with no locks needed. This is the one place the node deliberately reaches for threads, fenced off behind a Deferred. (The comment on `pow_thread_pool` names the use: resolving proof-of-work when sending a transaction, so the API call does not block the reactor while it hashes.) The proof-of-work machinery that uses this pool is detailed in **Chapter 37**.

16.7 Why Twisted and not the alternatives

This is the mandatory trade-off discussion. The honest version requires acknowledging that the obvious modern alternative — Python's own `asyncio` — did not exist when Twisted was chosen, and that Hathor has built a bridge toward it.

Twisted vs. `asyncio`

Python's standard library now ships `asyncio`: an event loop, futures, and the `async / await` syntax, built into the language. It covers much of the same ground as Twisted. So why is `hathor-core` a Twisted program?

The case for Twisted:

- **Maturity and battle-testing.** Twisted is over two decades old. Its TCP, TLS, and protocol implementations have run in production at scale for years. `asyncio` is younger and, especially in its early years, had rougher edges and fewer ready-made protocol implementations.
- **It predates `asyncio` by a decade-plus.** When the architecture was laid down, Twisted was the answer for asynchronous networking in Python; `asyncio` was not yet a credible option. Much foundational code, and the developers' fluency, grew up Twisted-shaped. Rewriting a working node's entire async core onto a different loop is a large, risky change with little user-visible benefit.
- **A complete framework, not just a loop.** Twisted bundles protocol abstractions (Protocol/Factory), `LoopingCall`, thread-pool integration, endpoints, and more as one coherent toolkit. `asyncio` gives you the loop and the primitives; you assemble more of the rest yourself.

The case for `asyncio`:

- **It is standard.** No third-party dependency; `async / await` is built into the language and is what most Python programmers now learn first. New contributors are more likely to know `asyncio` than Twisted's `Deferred / @inlineCallbacks` vocabulary.
- **Cleaner native syntax.** `async / await` is more ergonomic than `@inlineCallbacks + yield`, and the wider ecosystem increasingly targets `asyncio`.

Hathor's actual position — and a verified current reality. Hathor has not picked one and burned the bridge. It keeps the reactor behind the `hathor/reactor/` abstraction precisely so the kind of reactor can be chosen at boot. `initialize_global_reactor` takes a `use_asyncio_reactor` flag, and when set, it installs Twisted's **`asyncio reactor`** — a Twisted reactor whose event loop is, underneath, Python's `asyncio` loop (`hathor/reactor/reactor.py:44`):

```
def initialize_global_reactor(*, use_asyncio_reactor: bool = False) -> ReactorProtocol:
    ...
    if use_asyncio_reactor:
        import asyncio
        from twisted.internet import asyncioreactor
        from twisted.internet.error import ReactorAlreadyInstalledError
        try:
            loop = asyncio.get_running_loop()
        except RuntimeError:
            loop = asyncio.new_event_loop()
            asyncio.set_event_loop(loop)
        try:
            asyncioreactor.install(loop)
        except ReactorAlreadyInstalledError as e:
            ...
    from twisted.internet import reactor as twisted_reactor
    ...
```

This is a best-of-both arrangement: the code stays written against Twisted's APIs (Deferreds, Protocols, `LoopingCall`), but the underlying loop can be Python's `asyncio`. The option is exposed on the command line as `--x-asyncio-reactor` (`hathor_cli/run_node.py:161`), and the boot path threads the flag straight through to `initialize_global_reactor` (`run_node.py:202`).

Two honest caveats about the current reality, both visible in the code:

- **The `x_` prefix means experimental.** The flag is `--x-asyncio-reactor`. In this codebase the `x- / --x-` convention marks an unstable, opt-in feature. The default path is the standard Twisted reactor; the `asyncio` backend is a deliberate, off-by-default option, not the recommended mode. So the production answer remains "Twisted (default reactor)," with `asyncio` as an experimental alternative.
- **The default is the plain Twisted reactor.** When `use_asyncio_reactor` is false, the function skips the whole `asyncio` block and imports and returns Twisted's already-installed global reactor (`reactor.py:80`), casting it to Hathor's `ReactorProtocol` type. There is even a defensive comment in the code: if an `asyncio` reactor cannot be installed because some import already installed the default reactor indirectly, it raises with a pointed explanation (`reactor.py:70`). (This is a real footgun: importing from `hathor` at module level in a non-`RunNode` CLI tool can install the default reactor before `initialize_global_reactor` runs — which is also why both reactor functions carry the warning "must NOT be called at module level.")

The typed-reactor abstraction (why the wrapper exists at all)

There is a second motive for `hathor/reactor/` beyond the `asyncio` swap, and it ties back to Chapter 5. Twisted's reactor is typed loosely — its capabilities are spread across several `zope.interface`⁶ interfaces (`IReactorCore`, `IReactorTime`, `IReactorTCP`),

which static type-checkers like `mypy` do not follow well. Hathor defines a Python `Protocol`⁷ type that stitches those interfaces into one named type the checker does understand (`hathor/reactor/reactor_protocol.py:22`):

```
class ReactorProtocol(
    ReactorCoreProtocol,
    ReactorTimeProtocol,
    ReactorTCPProtocol,
    Protocol,
):
    """A Python protocol that represents the intersection of Twisted's
    IReactorCore+IReactorTime+IReactorTCP interfaces."""
    pass
```

Each piece stubs one Twisted interface — the time slice declares `seconds()`, `callLater()`, and `getDelayedCalls()` (`hathor/reactor/reactor_time_protocol.py:31`); the core slice declares `run()`, `stop()`, `callWhenRunning()`, and `addSystemEventTrigger()` (`hathor/reactor/reactor_core_protocol.py:38`); the TCP slice declares `listenTCP()` and `connectTCP()` (`hathor/reactor/reactor_tcp_protocol.py:33`). At initialization the code asserts the real reactor genuinely satisfies all three Twisted interfaces with `verifyObject`, then casts it to `ReactorProtocol` (`hathor/reactor/reactor.py:82`):

```
assert verifyObject(IReactorTime, twisted_reactor) is True
assert verifyObject(IReactorCore, twisted_reactor) is True
assert verifyObject(IReactorTCP, twisted_reactor) is True
_reactor = cast(ReactorProtocol, twisted_reactor)
```

So the wrapper buys two things at once: a single, mypy-legible type for "the reactor," and the seam where an alternative (asyncio-backed) reactor can be slotted in. The reactor abstraction itself gets a short dedicated chapter — **Chapter 23** — which builds on this section rather than repeating it.

Recap — Protocol the typing tool vs. Protocol the Twisted class (full treatment in Ch. 5 and §16.6). Confusingly, "Protocol" names two unrelated things here. Twisted's `Protocol` is the connection-handler class of §16.6. Python's `typing.Protocol` is a static-typing tool — "structural typing," i.e. any object with the right methods qualifies, no inheritance required. `ReactorProtocol` uses the typing one to describe the reactor's shape. → full treatment in Ch. 5 (typing) and §16.6 (Twisted Protocol).

And the thread question

The other "alternative" is the one Chapter 2 already settled: why a single-threaded event loop at all, instead of one OS thread per connection? Because a node holds thousands of mostly-idle connections; a thread each would burn memory and invite the race conditions and locking bugs that shared-memory threading is infamous for, and CPython's GIL⁸ would blunt any CPU parallelism anyway. The event-loop model gives

high I/O concurrency on one thread with almost no locking — and reserves real threads for the rare CPU-bound job, via the pool of §16.6.4. Twisted is the framework that makes that model practical in Python.

Recap

Twisted concept	Chapter-2 idea it realizes	Where in <code>hathor-core</code>
Reactor	the event loop	<code>hathor/reactor/</code> ; <code>run_node.py:202</code> , <code>:592</code>
<code>reactor.run()</code>	starting the loop (hand over the thread)	<code>run_node.py:592</code>
Deferred	a future / promise	<code>manager.py:23</code> ; <code>protocol.py:322</code>
<code>maybeDeferred</code> / <code>addErrback</code>	uniform result + error chaining	<code>protocol.py:342</code> , <code>:345</code> , <code>:347</code>
<code>@inlineCallbacks</code> (<code>yield</code>)	<code>async / await</code> (the yield point)	§16.5; cross-ref Ch. 4 Bridge
Protocol	per-event callbacks (<code>dataReceived</code> , ...)	<code>HathorProtocol</code> <code>protocol.py:51</code> , <code>:430</code>
Factory	the factory pattern (Ch. 3)	<code>factory.py:43</code> (<code>buildProtocol</code>) , <code>:55</code> , <code>:61</code>
<code>callLater</code>	one-shot timer	<code>protocol.py:247</code> (idle timeout)
LoopingCall	the periodic timer	<code>manager.py:251</code> , started <code>:331</code>
ThreadPool / <code>deferToThread</code>	the CPU escape hatch	<code>pow_thread_pool</code> <code>manager.py:234</code> , <code>:310</code>
asyncio reactor	swap the loop's backend	<code>reactor.py:44</code> (<code>--x-asyncio-reactor</code>)
<code>ReactorProtocol</code>	a typed name for the reactor (Ch. 5)	<code>reactor_protocol.py:22</code> ; <code>reactor.py:82</code>

Twisted is the asynchronous engine the entire node runs on, and every one of its abstractions is a Chapter-2 idea wearing a Twisted name: the **reactor** is the event loop, the **Deferred** is the future, `@inlineCallbacks` is `async / await` , the **Protocol**'s methods are the event callbacks, and the **thread pool** is the never-block rule's escape

hatch. Hathor wraps reactor access in `hathor/reactor/` for two reasons — a type the checker can follow, and a seam where an experimental `asyncio`-backed reactor can be installed via `--x-asyncio-reactor` (default off; standard Twisted reactor in production). From here, three threads of the book pick these names back up: the **reactor abstraction** gets its short dedicated treatment in **Chapter 23**; the **Protocol/Factory** pattern returns in full, with the handshake and peer state machine, in the **P2P chapters 34-35**; and **Deferreds** and `@inlineCallbacks` recur quietly throughout Part II wherever the node does something that finishes later. With Twisted understood, you can now read the node not as a script that runs but as a loop that reacts.

1. A blocking call does not return control to your program until it has finished; while it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). Full treatment in Ch. 2. ↩
2. I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. A node is overwhelmingly I/O-bound. Full treatment in Ch. 2. ↩
3. A Twisted Failure is an object that captures an exception together with its traceback, so an error can be carried along a Deferred's errback chain (and inspected later) rather than being raised and lost. Think of it as "an exception, boxed for asynchronous travel." ↩
4. Proof-of-work (PoW) is a scheme where producing a valid block (or transaction) requires finding a number that makes its hash fall below a target — hard to do, trivial to check. The hashing is CPU-bound, which is why Hathor runs it on a thread pool. Full treatment in Ch. 9. ↩
5. A thread pool is a managed set of reusable worker threads. Jobs are submitted to the pool rather than spawning a new thread each time; used to run blocking or CPU-bound work off the main event loop. Full treatment in Ch. 2. ↩
6. `zope.interface` is a library that lets Python code declare interfaces — named sets of methods a class promises to provide — and verify at runtime that an object implements one. Twisted uses it to describe the reactor's capabilities (`IReactorCore` , `IReactorTime` , `IReactorTCP`). It is an older, runtime-checked cousin of `typing.Protocol` . ↩
7. Python's `typing.Protocol` enables structural typing: any object that has the required methods/attributes satisfies the protocol, with no need to inherit from it. It is a static-typing tool, checked by `mypy` , and is unrelated to Twisted's connection-handler `Protocol` class. Full treatment in Ch. 5. ↩
8. The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel — a reason the ecosystem leans on the single-threaded event-loop model. Full treatment in Ch. 2. ↩